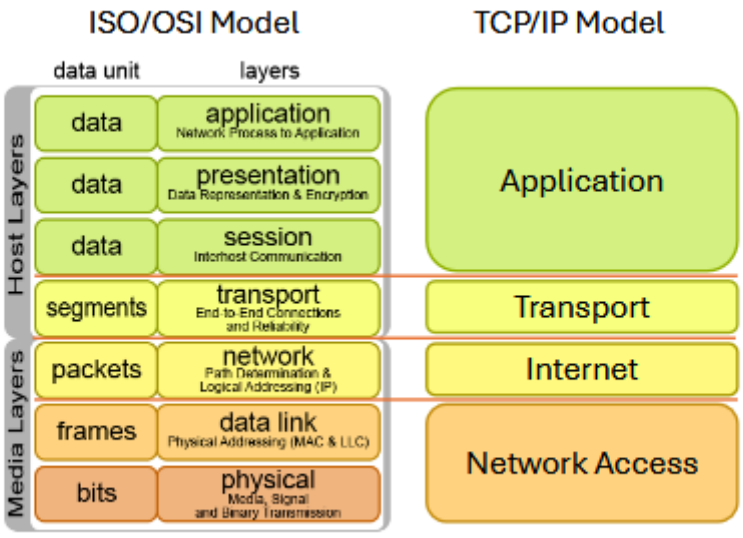
Modello TCP-IP

ISO/OSI Model vs TCP/IP Model

Il modello TCP/IP è un framework essenziale utilizzato per standardizzare e garantire una comunicazione dati affidabile attraverso reti interconnesse moderne, incluso Internet.
Difatti è progettato per garantire uno scambio dati **standardizzato, affidabile e scalabile** tra sistemi eterogenei.

Struttura del modello TCP/IP

È composto da **4 livelli**, ciascuno responsabile di funzioni specifiche nel processo di trasmissione dei dati.



1. **Application Layer** – **fornisce i protocolli per applicazioni e servizi di rete (HTTP, FTP, SMTP, DNS, ecc.)**.
2. **Transport Layer** – **gestisce la comunicazione tra processi con protocolli come TCP (affidabile, orientato alla connessione) e UDP (veloce, non affidabile)**.
3. **Internet Layer** – **si occupa dell'indirizzamento e dell'instradamento dei pacchetti tramite l'IP**.
4. **Network Access (Link) Layer** – **definisce come i dati vengono trasmessi fisicamente sulla rete** (Ethernet, Wi-Fi, ARP).

Confronto con il modello ISO/OSI

Guardando l'immagine soprastante possiamo notare come i 4 livelli del TCP/IP coprono le stesse funzionalità dei 7 livelli del modello ISO/OSI.

> **Nota:**
>
> Il modello ISO/OSI è teorico e quindi il modello TCP/IP pur essendo più semplice (solo 4 livelli) copre le stesse funzionalità in modo più pratico e integrato

- **Application (TCP/IP)** ≈ Livelli 5-6-7 (Application, Presentation, Session) di ISO/OSI.
- **Transport (TCP/IP)** ≈ Transport (ISO/OSI).
- **Internet (TCP/IP)** ≈ Network (ISO/OSI).
- **Network Access (TCP/IP)** ≈ Data Link + Physical (ISO/OSI).

> **Caratteristiche dei due modelli**
>
> - **TCP/IP: è lo standard de facto nelle reti moderne, scelto per la sua robustezza, flessibilità e diffusione.**
> - **ISO/OSI: rimane utile come modello concettuale e didattico, in quanto più dettagliato e modulare.**

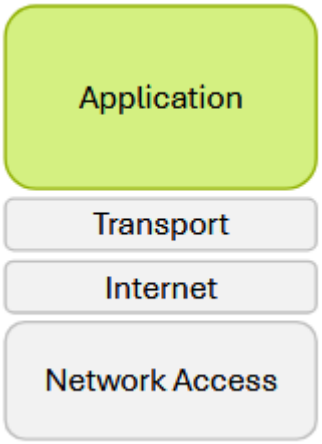
Application layer

Come anticipato, corrisponde ai livelli Application, Presentation e il Session del modello ISO/OSI.
Questo livello è il più vicino all'utente ed è responsabile della gestione dei protocolli che permettono alle applicazioni di comunicare in rete.

Caratteristiche

- **Fornisce servizi di rete direttamente alle applicazioni utente.**
- **Gestisce i protocolli a livello applicativo.**

Protocolli principali



Application Layer.png

- **DNS (Domain Name System):**
  - **risoluzione dei nomi di dominio**.
- **HTTP/HTTPS (HyperText Transfer Protocol):**
  - **navigazione web**.
- **FTP (File Transfer Protocol):**
  - **trasferimento di file**.
- **SMTP/POP3/IMAP:**
  - **protocolli per l'invio e la ricezione di e-mail**.
- **DHCP (Dynamic Host Configuration Protocol):**
  - **gestione e assegnazione dinamica degli indirizzi IP.**

Domain Name System (DNS)

Il **Domain Name System (DNS) è un sistema di denominazione gerarchico e decentralizzato che traduce i nomi di dominio leggibili dall'uomo (es. `www.cisco.com`) in indirizzi IP leggibili dalle macchine (es. `198.133.219.25`).**

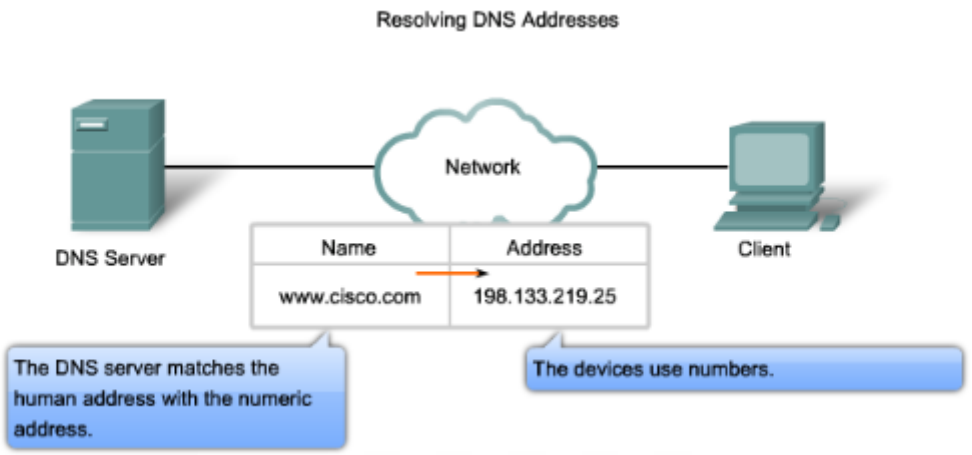
In pratica, quando vogliamo visitare un sito web ci basta ricordare il nome di dominio, mentre il DNS si occupa di tradurlo nell'indirizzo IP corrispondente, necessario per stabilire la connessione.

> **Esempio**
>
> `www.cisco.com` → `198.133.219.25`

Seguendo l'esempio fornito: all'utente serve il nome di dominio (`www.cisco.com`)
Il DSN lo traduce in indirizzo IP corrispondente, necessario per stabilire la

connessione.
Questo perché il nome si compone di domini:

- `.com` è il **Top-Level Domain (TLD)**
- `cisco` è il **Second-Level Domain**, cioè un sotto-dominio del TLD `.com`.



Come funziona il DNS (passaggi principali)

Il DNS è un **componente critico di Internet**: senza di esso gli utenti sarebbero costretti a ricordare solo gli indirizzi IP.
Supponiamo di voler navigare su un sito web: apriamo il browser → componiamo l'URL→ premiamo invio:
Il flusso prevede **6 passaggi** che sono i seguenti:

1. **DNS Query:**
   - **l'utente inserisce un nome di dominio nel browser**.
2. **Recursive Resolver:**
   - **la query viene inviata a un DNS resolver, che avvia il processo di risoluzione.**
3. **Root Server:**
   - **il resolver interroga un root server per ottenere l'indicazione del server TLD corretto (es. `.com`, `.org`)**.
4. **TLD Server:**
   - **il resolver interroga il server TLD per sapere quale sia il server DNS autorevole del dominio richiesto**.
5. **Authoritative DNS Server:**
   - **restituisce l'indirizzo IP associato al dominio**.
6. **Response:**
   - **il resolver invia l'IP al browser, che stabilisce la connessione con il web server.**

> ⓘ **Info**
>
> Esistono server DNS pubblici, ad esempio quelli di **Google (8.8.8.8, 8.8.4.4)** o **Cloudflare (1.1.1.1)**.

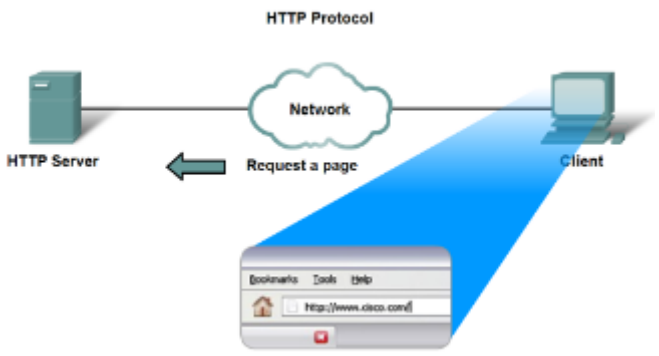
HyperText Transfer Protocol (HTTP)

È il protocollo utilizzato per la navigazione web:

- **permette lo scambio di dati tra un client (il browser) e un server (il sito web)**.
- Esiste anche la variante sicura - **HTTPS (HyperText Transfer Protocol Secure):**
- **è un'estensione di HTTP**.
- **Essa aggiunge un livello di sicurezza grazie alla crittografia, proteggendo i dati scambiati tra client e server**.

Come funziona la navigazione web

La navigazione web avviene tramite **6 passaggi:**



HTTP - Web Browsing Process.png

1. **Digitazione dell'URL**:
   - **l'utente inserisce l'indirizzo di un sito nella barra del browser.**
2. **DNS Resolution**:
   - **il browser invia una query DNS per tradurre l'URL in un indirizzo IP.**
3. **Connessione al server**:
   - **il browser stabilisce una connessione con il server corrispondente.**
4. **Invio della richiesta**:

- **tramite HTTP o HTTPS, il browser invia una richiesta (in genere di tipo `GET`) al server, solitamente per ottenere il documento predefinito (ad esempio `index.html`).**

5. **Ricezione della risposta**:
   - **il server invia al browser il contenuto HTML della pagina richiesta.**
6. **Rendering della pagina**:
   - **il browser interpreta il codice HTML (insieme a eventuali CSS e JavaScript) e visualizza la pagina formattata.**

File Transfer Protocol (FTP)

Il File Transfer Protocol (FTP) è un protocollo standard, connection-oriented, utilizzato per il trasferimento affidabile di file tra un client e un server su una rete basata su TCP (es. Internet).

Fasi principali

Le fasi di questo livello del modello TCP/IP sono conseguenziali:

1. **Stabilimento della connessione**
   - **Control Connection**: **il client apre una connessione al server per inviare comandi (porta 21)**.
   - **Data Connection**: **viene aperta una seconda connessione separata per trasferire effettivamente i file (porte variabili)**.
2. **Autenticazione**
   - **L'utente inserisce username e password.**
   - **Per file pubblici, spesso si utilizza la modalità anonymous FTP, con `anonymous` come username e `guest` come password predefinita.**
3. **Trasferimento del file**
   - **I comandi (es. `RETR` per scaricare, `STOR` per caricare) viaggiano sulla control connection.**
   - **I dati veri e propri vengono trasferiti sulla data connection**.

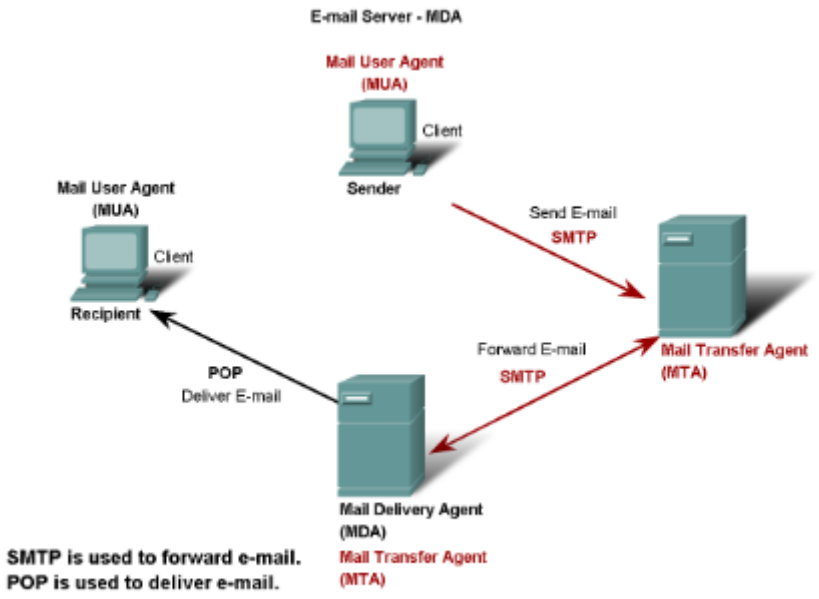
SMTP/POP3/IMAP

Quando parliamo di posta elettronica, entrano in gioco tre protocolli principali che collaborano per permettere l'invio e la ricezione dei messaggi:

1. **SMTP (Simple Mail Transfer Protocol)**:
   - protocollo di invio della posta

2. **[[#**POP3 (Post Office Protocol v3)**|POP3 (Post Office Protocol v3)]]:**
   - ricezione della posta con download locale
3. **[[#**IMAP (Internet Message Access Protocol)**|IMAP (Internet Message Access Protocol)]]:**
   - icezione della posta mantenendo i messaggi sul server.

Spieghiamo in dettaglio questi tre protocolli usando questa immagine:
L'immagine illustra il flusso di comunicazione tra client e server.



SMTP-POP3-IMAP.png

Cos'è il protocollo SMTP (Simple Mail Transfer Protocol)

- È il protocollo che gestisce **l'invio delle e-mail**.
- Opera in due scenari distinti:
  1. **Dal client al server di posta** → quando l'utente preme "Invia", il client (MUA, Mail User Agent) trasmette il messaggio al server (MTA, Mail Transfer Agent).
  2. **Da server a server** → gli MTA possono scambiarsi il messaggio fino a raggiungere il server del destinatario.
- **Porte utilizzate:**
  - 25 (standard per inoltro server-server)
  - 587 (invio autenticato, consigliato).

Nell'immagine:

- Il **Mail User Agent (MUA)** del mittente (es. Outlook, Thunderbird, Gmail client) invia la mail tramite **SMTP** al **Mail Transfer Agent (MTA)**.
- Questo MTA può inoltrarla a più server intermedi finché non raggiunge l'MTA del destinatario.

POP3 (Post Office Protocol v3)

- **È il protocollo che gestisce la ricezione della posta.**
- **Funzionamento tipico:**
  - Il client scarica i messaggi dal server.
  - Dopo il download, i messaggi vengono normalmente eliminati dal server.
  - Di conseguenza, la posta resta **solo sul dispositivo locale**.
- Porta tipica: **110** (oppure **995** se si utilizza una connessione sicura SSL/TLS). Nell'immagine:
- Il secondo **Mail Transfer Agent (MTA)** a cui è stata inoltrata l'email del mittente tramite il primo MTA diventa un **Mail Delivery Agent (MDA)**, ovvero consegna la mail al destinatario Mail User Agent (MUA).
- Il MUA del destinatario si connette al server e, tramite POP3, **scarica definitivamente il messaggio**.

× **Svantaggio**

non è possibile mantenere i messaggi sincronizzati su più dispositivi.

IMAP (Internet Message Access Protocol)

-
  - È una **versione più moderna e flessibile** del protocollo POP3.
- Caratteristiche principali:
  - I messaggi **rimangono memorizzati sul server**.
  - L'utente può visualizzarli, spostarli e organizzarli come se fossero locali.
  - Permette l'accesso allo stesso account da più dispositivi (PC, smartphone, tablet) mantenendo tutto sincronizzato.
- **Porta tipica:** 143 (oppure 993 se si usa SSL/TLS).

Nell'immagine:

- Non è mostrato esplicitamente il flusso IMAP, ma il comportamento è analogo a POP3.
- Differenza fondamentale: i messaggi **non vengono cancellati dal server**, così l'utente può consultarli e gestirli in parallelo da più dispositivi.

🔗 **In sintesi**

- **SMTP = invio / inoltro di email**
- **POP3 = ricezione con download locale (senza sincronizzazione)**
- **IMAP = ricezione mantenendo le mail sul server e sincronizzate tra più dispositivi**

DHCP (Dynamic Host Configuration Protocol)

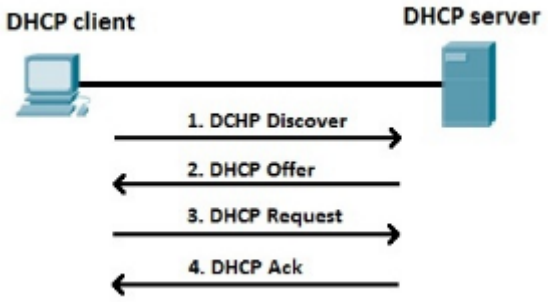
Il **DHCP** è:

- **il protocollo che si occupa di assegnare automaticamente un indirizzo IP e altri parametri di rete (gateway, DNS, subnet mask, ecc.) a un dispositivo quando questo si collega a una rete.**
  Il processo avviene in più fasi:

1. **DHCP Discover:**
   - **Quando un nuovo dispositivo entra in rete, non conosce l'indirizzo del server DHCP (spesso integrato nel router). Per questo invia un messaggio broadcast chiamato *DHCPDISCOVER* per cercare un server disponibile**.
2. **DHCP Offer:**
   - Il server DHCP che riceve la richiesta risponde con un ***DHCPOFFER***
   - **DHCPOFFER: una proposta che contiene un indirizzo IP libero e le configurazioni di rete necessarie**.
3. **DHCP Request:**
   - **Il client, dopo aver ricevuto una o più offerte, sceglie quella da accettare e invia un *DHCPREQUEST* al server, comunicando la sua decisione**.
4. **DHCP Acknowledgment:**
   - **Infine, il server conferma l'assegnazione inviando un *DHCPACK* (acknowledgment).**

- A questo punto il client può iniziare a usare l'indirizzo IP e le configurazioni ricevute.

Per comprendere meglio questi passaggi prendiamo come esempio questa immagine:



DHCP.png

Spiegazione:

- Quando un **DHCP Client** (ad esempio un laptop appena acceso) entra in una rete, non conosce l'indirizzo del server DHCP.
  - Per questo invia un **messaggio broadcast** **DHCP Discover** sulla rete locale, chiedendo se esista un server in grado di fornirgli un indirizzo IP.
- Il **DHCP Server** (spesso coincide con il router domestico o aziendale) riceve la richiesta e risponde con un **DHCP Offer**, cioè una proposta che contiene:
  - un indirizzo IP libero,
  - la subnet mask,
  - il gateway predefinito,
  - e altri parametri come i server DNS.
- Il client, dopo aver ricevuto l'offerta, invia un **DHCP Request**, dichiarando esplicitamente che intende accettare quell'assegnazione di IP da quel server.
- Infine, il server invia un **DHCP ACK (Acknowledgment)**, confermando la validità dell'assegnazione. Da questo momento, il client può utilizzare l'indirizzo IP e i parametri ricevuti per comunicare in rete.

> ☰ **Esempio Reale:**
>
> Immaginiamo che tu arrivi in un bar con il tuo smartphone e ti connetta al Wi-Fi gratuito:
>
> 1. Il tuo telefono non sa quale IP usare → invia un **DHCP Discover**.
> 2. Il router del bar riceve il messaggio e risponde con un **DHCP Offer**, proponendo ad esempio l'IP `192.168.1.25`.

3. Il telefono accetta quell'indirizzo → invia un **DHCP Request**.
4. Il router conferma con un **DHCP ACK**. Ora il tuo telefono ha un IP valido (`192.168.1.25`), sa quale gateway usare (`192.168.1.1`) e quali DNS contattare (ad esempio `8.8.8.8`).

Analogia: ›
Il DHCP funziona praticamente come un "contratto temporaneo" tra un client (host) e un server (tipicamente un router o un server dedicato) per l'assegnazione di un indirizzo IP e altre informazioni di rete.
Se lo traduciamo in termini di contratto:

1. **DHCP Discover → Richiesta di contratto**
   Il client "dichiara" che vuole entrare nella rete e cerca un server che possa fornirgli un IP.

   > **È come dire:** ***"C'è qualcuno disposto a stipulare un contratto di indirizzo con me?"***

2. **DHCP Offer → Offerta di contratto**
   Il server risponde con i termini del contratto: un IP libero, subnet mask, gateway, tempo di lease, ecc. È l'equivalente della proposta contrattuale che il server fa al client.
3. **DHCP Request → Accettazione del contratto**
   Il client accetta l'offerta specifica, indicando chiaramente quale server e quale IP vuole usare. In pratica, accetta le condizioni proposte.
4. **DHCP ACK → Conferma finale del contratto**
   Il server conferma ufficialmente l'assegnazione dell'indirizzo IP e degli altri parametri al client. A questo punto, il "contratto" è valido per il periodo di lease indicato.

> Analogicamente, puoi anche vedere il **DHCP Lease** come la durata del contratto: scaduto il tempo, se il client vuole continuare a usare lo stesso IP, deve rinnovare il contratto (DHCP Request/Renew e DHCP ACK/Renew).

Transport Layer

Il Transport Layer è il secondo livello del modello TCP/IP (l'equivalente del Transport Layer nel modello ISO/OSI).

Funzioni principali

- Fornisce **trasmissione dati** tra host.
- Può essere **affidabile(protocollo TCP)** o **non affidabile(protocollo UDP)**, a seconda del protocollo utilizzato.
- Include meccanismi per assicurare che i dati arrivino correttamente e nell'ordine giusto (solo se il protocollo scelto è affidabile).

Come anticipato all'inizio del file, la comunicazione tra processi avviene tramite **2 protocolli:**

Protocollo	Tipo di comunicazione	Caratteristiche principali
TCP	Connection-oriented(orientato alla connessione)	Affidabile; garantisce che i dati arrivino integri e nell'ordine corretto; usa meccanismi come **acknowledgment**, **sequencing** e **flow control**.
UDP	Connectionless (senza connessione)	Non affidabile; i dati possono arrivare fuori ordine o andare persi; basso overhead, ideale per streaming o applicazioni real-time.

TCP (Transmission Control Protocol)

Tipo: Connection-oriented (orientato alla connessione)
Funzione principale: Trasmissione dati affidabile tra host.

Caratteristiche principali:

- **Affidabilità:** garantisce che i dati arrivino corretti e nell'ordine giusto.
- **Error checking:** controlla la presenza di errori nei pacchetti.
- **Flow control:** regola il flusso di dati tra mittente e destinatario per evitare sovraccarichi.
- **Acknowledgment:** conferma la ricezione di pacchetti, evitando perdite o duplicazioni.

Sintesi: TCP assicura che la comunicazione sia affidabile e ordinata, rendendolo ideale per applicazioni come web, email e trasferimento file.

Spiegazione dell'immagine: il protocollo TCP in azione

Il TCP, essendo **connection-oriented**:
- **richiede di instaurare una connessione logica tra i due host (Host A e Host B) prima di scambiare dati veri e propri.**

Allo stesso modo, quando la comunicazione è finita, la connessione viene chiusa in modo ordinato.
Come possiamo vedere dall'immagine sottostante nel protocollo TCP vi è prima una creazione della connessione (Connection Establishment) e in seguito una terminazione della connessione quando finisce (Connection Termination):

1. TCP Connection Establishment (Three-Way Handshake)

- Questo è il "rito di introduzione" tra due dispositivi.
- **Serve per sincronizzare i numeri di sequenza iniziali (Sequence Number - SYN e Acknowledgment Number - ACK) e negoziare i parametri della connessione.**
  Fase per Fase:

1. `Host A --> Host B: SYN`
   - **L'Host A (il client, ad esempio il tuo browser) invia all'Host B (il server, ad esempio google.com) un pacchetto speciale chiamato SYN (Synchronize).**
   - In pratica, questo pacchetto dice: *"Ciao Server, vorrei avviare una connessione. Il mio numero di sequenza iniziale è X."*
2. `Host B --> Host A: SYN-ACK`
   - **L'Host B riceve il SYN. Se è pronto ad accettare la connessione, risponde con un unico pacchetto che è sia SYN che ACK (Acknowledgment)**.
   - Questo messaggio significa: *"Ok Client, ho ricevuto la tua richiesta (ACK) e sono d'accordo ad aprire la connessione. Il _mio numero di sequenza iniziale è Y."*
3. `Host A --> Host B: ACK`
   - **Infine, l'Host A invia un ACK al server per confermare la ricezione del SYN-ACK**.

- Dice: *"Perfetto Server, ho ricevuto la tua conferma. La connessione è ora stabilita."*

Dopo questo terzo scambio, i dati possono iniziare a fluire in modo affidabile tra i due host.

2. TCP Connection Termination (Four-Way Handshake)

Quando una delle due parti ha finito di inviare dati;

- **avvia la procedura per chiudere la connessione in modo elegante e senza perdite.**
  Questo processo **è simmetrico:**
- **cioè ogni lato deve chiudere la sua metà della connessione indipendentemente.**

Fase per Fase:

1. `Host A --> Host B: FIN`
   - **L'Host A (che ha finito di inviare dati) invia un pacchetto FIN (Finish) all'Host B.**
   - Comunica: *"Ho finito di inviarti dati. Vorrei chiudere la connessione dalla mia parte."*

> **Nota:**
>
> - **Host A non può più inviare dati, ma può ancora riceverne da Host B.**

2. `Host B --> Host A: ACK`
   - **L'Host B riceve il FIN e risponde immediatamente con un ACK.**
   - Dice: *"Ok, ho ricevuto la tua richiesta di chiusura."*
   - **A questo punto, Host B sa che non riceverà più dati da Host A, ma potrebbe averne ancora da inviare.**
3. `Host B --> Host A: FIN`
   - **Quando anche Host B ha finito di inviare tutti i suoi dati rimanenti ad Host A, invia a sua volta un pacchetto FIN.**
   - Comunica: *"Anche io ho finito. Chiudiamo definitivamente."*
4. `Host A --> Host B: ACK`

- **Host A riceve il FIN e invia un ultimo ACK per confermare.**
- **Dopo un periodo di time-out per essere sicuro che l'ACK sia arrivato, entrambi gli host rilasciano completamente le risorse della connessione.**

Esempio Reale: caricare questa pagina web

Immagina di aver digitato `www.google.com` nella barra del browser e premuto Invio.

1. **Stabilizione della Connessione (Three-Way Handshake):**

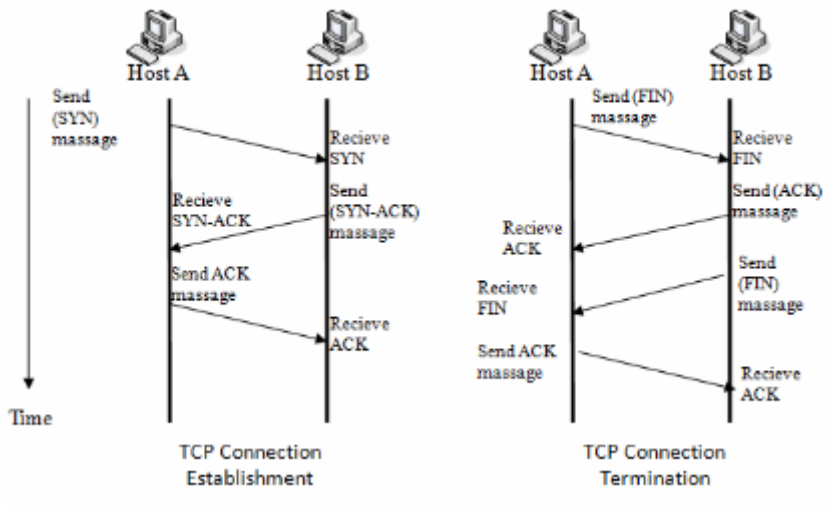
- **Il tuo computer (Host A) invia un pacchetto SYN al server di Google (Host B).**
- **Il server di Google risponde con un SYN-ACK.**
- **Il tuo computer conclude con un ACK.**
- **Risultato:** È stato costruito un "tubo" affidabile tra te e Google. Ora il tuo browser può inviare la richiesta HTTP per la homepage attraverso questa connessione TCP appena aperta.

2. **Trasferimento Dati:**

- Attraverso la connessione stabilita, il server di Google invia i dati della pagina web (HTML, CSS, immagini). TCP si assicura che tutti i pacchetti arrivino, siano in ordine e senza errori, usando i meccanismi di ACK e ritrasmissione.

3. **Terminazione della Connessione (Four-Way Handshake):**

- **Dopo che la pagina è stata caricata e non c'è più attività per un po', il tuo browser (Host A) invia un FIN per dire che ha chiuso la sua finestra**.
- **Il server di Google conferma con un ACK.**
- **Poco dopo, anche il server di Google (Host B), avendo inviato tutti i dati, invia il suo FIN.**
- **Il tuo computer risponde con un ultimo ACK.**
- **Risultato: La connessione è chiusa in modo pulito e le risorse di rete su entrambi i lati vengono liberate.**



TCP Protocol.png

TCP Header Structure

L'intestazione TCP contiene diversi campi fondamentali per il suo funzionamento affidabile.

TCP Header Structure

Field	Length (bits)	Description
Source Port	16	Port number of the sending application
Destination Port	16	Port number of the receiving application
Sequence Number	32	Position of the first byte of data in the segment
Acknowledgment Number	32	Next expected byte from the sender
Data Offset	4	Size of the TCP header in 32-bit words
Reserved	3	Reserved for future use, must be zero
Flags	9	Control flags (e.g., SYN, ACK, FIN)
Window Size	16	Size of the sender's receive window
Checksum	16	Error-checking of the header and data
Urgent Pointer	16	Points to urgent data (if URG flag is set)
Options	Variable	Optional fields for additional control
Padding	Variable	Ensures header length is a multiple of 32 bits

KEY FIELD

Source and Destination Port:
Identifies the sending and receiving applications.

Sequence Number:
Tracks the position of data in the segment.

Acknowledgment Number:
Confirms receipt of data.

Checksum:
For error-checking of header and data

Flags:
- SYN: Synchronize sequence numbers (connection setup).
- ACK: Acknowledgment field significant.
- FIN: No more data from sender (connection teardown).

TCP Header Structure.png

Guardando questa immagine possiamo notare diversi campi della struttura dell'intestazione TCP, i principali sono:

1. Porte (Source Port & Destination Port - 16 bit ciascuna)

- **Funzione:**
  - **Identificano rispettivamente l'applicazione sul mittente e sul destinatario**.
  - **Indirizzano il segmento al processo corretto su ogni host**.

2. Sequence Number (Numero di Sequenza - 32 bit)

- **Funzione:**
  - **Indica la posizione del primo byte di dati nel segmento corrente all'interno del flusso totale di byte.**
  - **È cruciale per riordinare i segmenti ricevuti fuori ordine.**

3. Acknowledgment Number (Numero di Riscontro - 32 bit)

- **Funzione: Conferma la ricezione dei dati**
  - In sostanza indica **il prossimo byte che il ricevitore si aspetta di ricevere, confermando così che tutti i byte precedenti sono stati ricevuti correttamente.**

4. Checksum (16 bit)

- **Funzione:**
  - **Utilizzato per il controllo degli errori sull'header e sui dati.**
  - **Verifica l'integrità del segmento per rilevare eventuali corruzioni avvenute durante la trasmissione.**

5. Flags (Bandierine di Controllo - 9 bit)

- **Funzione: Bit di controllo che segnalano lo stato o l'intento della comunicazione**.
  Le tre flags principali sono:
  - **SYN (Synchronize):** Utilizzato nella **[[#**1. TCP Connection Establishment (Three-Way Handshake)**|fase di stabilimento della connessione (three-way handshake)]].**
  - **ACK (Acknowledgment):**
    - **Indica che il campo Acknowledgment Number è valido**.
    - **Viene impostato per confermare la ricezione di dati**.
  - **FIN (Finish):**
    - **Utilizzato per chiudere la connessione in modo ordinato, indicando che il mittente non ha più dati da inviare.**

UDP (User Datagram Protocol)

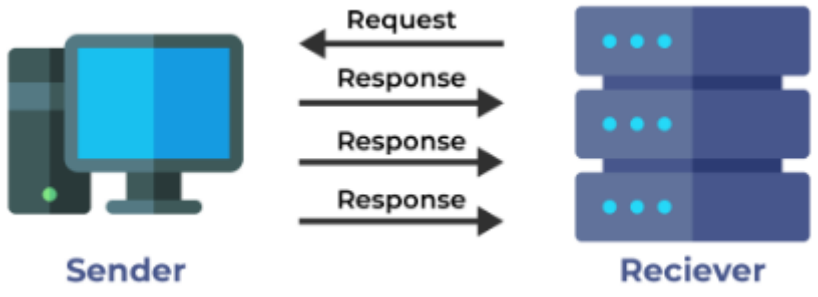
Protocollo di trasporto **connectionless** e **non affidabile**. A differenza di TCP, non garantisce la consegna, l'ordinamento o la ritrasmissione dei pacchetti.

- **Controllo Errori (Checksum):**

    - È presente un **checksum** nell'header **per rilevare errori di trasmissione nei dati e nell'indirizzo.**
    - **I pacchetti corrotti vengono scartati silenziosamente *senza* richiedere una ritrasmissione**.
    - **Questo meccanismo minimale è stato inserito per evitare usi impropri e prevenire la corruzione "silenziosa" dei dati a livello applicativo**, guidando così gli sviluppatori verso scenari d'uso appropriati.
- **Differenza con OSI:** A differenza dei servizi connectionless puri degli strati inferiori del modello ISO/OSI (es. Livello 2 - Data Link), il UDP opera a Livello 4 e fornisce comunque i servizi base del suo strato, come il multiplexing basato sulle porte e, appunto, un controllo opzionale degli errori.
- **Scenario d'uso:** Ideale dove **la velocità e la bassa latenza sono più importanti dell'affidabilità totale**, come nello streaming audio/video, nelle query DNS o nei giochi online.

Spiegazione dell'immagine

Per comprendere meglio come funziona il protocollo UDP, basta guardare questa immagine:



UDP Protocol.png

L'immagine mostra il principio fondamentale di UDP:

- **il mittente invia dati senza avere alcuna garanzia o conferma che arrivino a destinazione.**

1. **Request (Richiesta): Il Sender (mittente) invia uno o più datagrammi UDP verso il ricevitore.**
    - **Nessuna connessione:**
        - **Non c'è un handshake iniziale (nessun SYN, SYN-ACK, ACK come in TCP). Il mittente "spara" i dati nella rete e spera per il**

meglio.

- **Nessuna garanzia:**
  - **Ogni freccia "Request" rappresenta un datagramma che potrebbe perdersi, arrivare duplicato o fuori ordine. UDP non fa nulla per evitarlo.**

2. **Response (Risposta) *Opzionale*:** Le frecce "Response" sono la parte *cruciale* per capire l'immagine.
   In UDP, non esiste una "risposta" a livello di protocollo.
   - **Le "Response" nell'immagine non sono inviate dal protocollo UDP, ma dall'applicazione che sta usando UDP.**
   - **Se l'applicazione sul receiver ha bisogno di confermare la ricezione, deve farlo a suo modo, inviando a sua volta un datagramma UDP come risposta all'applicazione mittente**.
   - Ma attenzione: anche questa risposta dell'applicazione è un datagramma UDP **non affidabile!** Anche essa potrebbe perdersi.

In sintesi:

L'immagine mostra che la responsabilità di gestire la logica di comunicazione (conferme, ritrasmissioni, ordine) è **totalmente a carico dell'applicazione**. **UDP si limita a inviare e ricevere datagrammi, punto.**

Esempio Reale: Una Ricerca DNS

Scenario: Il tuo computer (client) deve risolvere il nome `www.google.com` in un indirizzo IP.

1. **Request (dal Client):**
   - La tua applicazione (il resolver DNS) prepara una domanda: "Qual è l'IP di `www.google.com`?".
   - **Usa UDP: Impacchetta questa domanda in uno o più datagrammi UDP e li invia all'indirizzo del server DNS. Il protocollo UDP manda i pacchetti e non si preoccupa altro.**
2. **Response (dal Server DNS *se tutto va bene*):**
   - **Il server DNS riceve la richiesta (se il datagramma non si è perso)**.
   - **Elabora la query e prepara la risposta: "L'IP è `142.251.209.36`"**.

- **Ancora UDP: Impacchetta questa risposta in un datagramma UDP e lo invia indietro all'indirizzo IP e alla porta del tuo computer.**
- **Il tuo computer riceve la risposta (se questo datagramma non si è perso) e la passa al browser.**

Cosa succede se qualcosa va storto?

- **Se la Request si perde: Il tuo computer non riceve alcuna risposta. Dopo un time-out stabilito dall'applicazione (non da UDP!), il tuo computer ritrasmetterà la stessa identica richiesta UDP.**
- **Se la Response si perde: Il tuo computer non riceve alcuna risposta. Anche in questo caso, dopo un time-out, la tua applicazione ritrasmetterà la richiesta.**

Perché usare UDP per questo?

Perché è velocissimo.
Inviare una singola richiesta e sperare in una singola risposta è molto più efficiente che avviare una complessa connessione TCP con handshake a tre vie per una transazione così piccola e veloce.
Quindi l'immagine mostra un flusso di datagrammi (richieste e risposte) dove il protocollo di base non fornisce alcuna garanzia intrinseca.
L'affidabilità, se necessaria, deve essere implementata a un livello superiore (dall'applicazione).

UDP Header Structure

L'header UDP è estremamente semplice e minimalista, soprattutto se confrontato con quello di TCP.
Questo riflette perfettamente la filosofia del protocollo:
- **essere leggero e veloce, delegando all'applicazione qualsiasi necessità di complessità.**

L'header è composto da soli **4 campi** per un totale di **8 byte** (64 bit).

1. Source Port (Porta Sorgente - 16 bit)

- **Funzione:**

- **Identifica il numero di porta dell'applicazione mittente sul dispositivo di origine.**

Nota:

- Questo campo è **opzionale**.
- Se non utilizzato, viene impostato a zero.
  - Viene usato quando il destinatario ha bisogno di sapere a quale porta rispondere.

2. Destination Port (Porta Destinazione - 16 bit)

- **Funzione:**
  - **Questo campo è obbligatorio.**
  - **Identifica il numero di porta dell'applicazione destinataria sul dispositivo di destinazione.**

Esempio:

Se un client vuole effettuare una query DNS, indirizzerà il suo datagramma UDP alla **Porta 53** del server, che è la porta standard per il servizio DNS.

3. Length (Lunghezza - 16 bit)

- **Funzione:**
  - **Specifica la lunghezza totale dell'intero datagramma UDP, comprendendo sia l'header (8 byte) che i dati (payload)**.

Calcolo:

Il valore minimo è 8 (se il datagramma non ha dati payload). Il valore massimo teorico è 65.535 byte, ma nella pratica è spesso limitato dalle reti sottostanti.

4. Checksum (Checksum - 16 bit)

- **Funzione:**

- **Utilizzato per il controllo degli errori sull'header, sui dati e su una "pseudo-header" IP (che include indirizzi IP sorgente e destinazione).**
- **Il suo scopo è rilevare se il datagramma è stato corrotto durante la trasmissione.**

> ⓘ **Comportamento:**
>
> **Se il checksum calcolato dal ricevitore non corrisponde, il datagramma viene scartato silenziosamente, senza alcuna notifica al mittente.**
> Questo è l'**unico** meccanismo di "controllo" offerto da UDP, ed è anche esso **opzionale** (se non usato, il campo è impostato a zero).

Internet Layer

Terzo e penultimo livello del modello TCP/IP.
Questo strato è il **cuore** di tutta Internet.
Il suo compito principale è:

- **permettere la comunicazione tra host su reti diverse, indirizzando e instradando i pacchetti attraverso più "salti" (router) dalla sorgente alla destinazione.**

Funzioni Principali:

1. **Indirizzamento Logico (Logical Addressing):**
   - Assegna a ogni dispositivo su una rete un indirizzo univoco e logico: l'**Indirizzo IP**.
   - Questo indirizzo (es. `192.168.1.10` o `2001:db8::1`) è gerarchico:
     - contiene informazioni sulla rete di appartenenza e sull'host specifico, permettendo un instradamento efficiente attraverso il globo.
2. **Instradamento (Routing):**
   - **È il processo di determinare il percorso migliore che un pacchetto deve fare attraverso una serie di router per passare da una rete di partenza a una rete di destinazione.**
   - I **router** sono i dispositivi che operano a questo livello, prendendo decisioni di instradamento basate su tabelle di routing.
3. **Consegna al Destinatario Corretto:**

- **Gestisce il forwarding dei pacchetti attraverso multiple reti, garantendo (o almeno, tentando di garantire) che raggiungano la rete di destinazione corretta.**

Protocolli Chiave

Questo livello per la comunicazione e il trasferimento di dati sfrutta 3 protocolli:

1. IP(Internet Protocol - IPv4 e IPv6):

- **Il protocollo principale.**
- È responsabile dell'incapsulamento dei dati in **pacchetti** (o datagrammi), dell'assegnazione degli indirizzi di sorgente e destinazione e del loro instradamento attraverso la rete.
- È un protocollo **connectionless** e **best-effort**: non stabilisce una connessione prima di inviare i dati e non garantisce la consegna (questo è compito degli strati superiori, come TCP).

2. ICMP (Internet Control Message Protocol)

- Il compito di questo protocollo è **scambiare messaggi di controllo e di errore** tra dispositivi di rete (host, router).
- È il "meccanismo di segnalazione" fondamentale di IP.

Scopo principale

- **Non trasporta dati applicativi** (come web, email, ecc.). Il suo unico compito è **creare report** e **diagnosticare** problemi relativi all'instradamento dei pacchetti IP.
- Viene utilizzato da **tool di diagnostica di rete** per verificare la connettività e le prestazioni.

Come funziona

- Quando un router o un host rileva un problema (es. un host non raggiungibile, un pacchetto che supera il suo "time to live"), **genera un messaggio ICMP** e lo invia **alla sorgente** del pacchetto originale per notificare l'errore.
- Esempio: Se un router non sa come raggiungere la rete di destinazione, scarterà il pacchetto e invierà un messaggio ICMP `Destination Unreachable (Type 3)` all'IP mittente per informarlo del problema.

Messaggi comuni del protocollo ICMP

L'ICMP (Internet Control Message Protocol) utilizza diversi "tipi" di messaggi per svolgere funzioni specifiche.

1. **Echo Request (Tipo 8) & Echo Reply (Tipo 0)**
   - **Funzione: Questi due messaggi lavorano in coppia per formare il cuore del comando `ping`.**
   - **Meccanismo:**
     1. **Un host invia un Echo Request (Tipo 8) a un altro host. È come bussare alla porta e chiedere "C'è nessuno?".**
     2. **Se l'host di destinazione è raggiungibile e operativo, risponde con un Echo Reply (Tipo 0), che significa "Sì, sono qui!"**.
   - **Scopo: Verificare la raggiungibilità di un host e misurare il tempo di andata e ritorno (round-trip time) della comunicazione**.
2. **Destination Unreachable (Tipo 3)**

- **Funzione: Segnalare che un pacchetto IP non ha potuto essere consegnato alla sua destinazione finale.**
- **Meccanismo: Un router o un host intermedio che incontra un problema durante l'inoltro di un pacchetto genera questo messaggio e lo invia alla sorgente del pacchetto originale.**
- **Scopo: Comunicare il motivo specifico del fallimento.**
  - All'interno del messaggio di Tipo 3, c'è un **codice** che specifica il problema:
    - **Code 0: Network Unreachable** → **La rete di destinazione non è raggiungibile** (es. non esiste una rotta nella tabella di routing).
    - **Code 1: Host Unreachable** → **La rete è raggiungibile, ma l'host specifico al suo interno non lo è** (es. spento o disconnesso).
    - **Code 3: Port Unreachable** → **L'host di destinazione è raggiungibile, ma nessuna applicazione è in ascolto sulla porta richiesta**.
      - Questo è molto comune.

3. **Time Exceeded (Tipo 11)**

- **Funzione: Segnalare che un pacchetto è stato scartato perché il suo TTL (Time to Live) è arrivato a zero**.

- **Meccanismo:** **Ogni router che inoltra un pacchetto decrementa il suo TTL di 1. Quando il TTL raggiunge 0, il router scarta il pacchetto e invia un messaggio Time Exceeded al mittente originale**.
- **Scopo:**
  1. **Prevenire loop di instradamento** infiniti.
  2. **Abilitare il comando** `traceroute` / `tracert`. Questo strumento invia deliberatamente pacchetti con TTL molto bassi (1, 2, 3, ...) per forzare i router lungo il percorso a generare questi messaggi e rivelare così la loro identità e il percorso.

4. **Redirect (Tipo 5)**

- **Funzione:** **Permettere a un router di "istruire" un host su una rotta migliore per raggiungere una certa destinazione.**
- **Meccanismo:**
  - **Su una rete locale, un host invia un pacchetto per una destinazione esterna al suo gateway predefinito.**
  - **Se il gateway sa che esiste un router sulla stessa rete locale che ha un percorso più diretto, inoltra il pacchetto ma invia anche un Redirect all'host.**
  - Il messaggio dice all'host: "Per raggiungere quella destinazione, la prossima volta invia il pacchetto direttamente a questo altro router, non a me".
- **Scopo:** **Ottimizzare l'efficienza del routing all'interno di una singola sottorete locale.**

3. Address Resolution Protocol ARP

L'**Address Resolution Protocol (ARP)** ha un solo compito:

- **risolvere (tradurre) un indirizzo IP (logico, di livello 3) in un indirizzo MAC (fisico, di livello 2) all'interno della stessa rete locale (LAN).**
- È il "elenco del telefono" della rete.

Come Funziona

Immagina che il **Tuo PC (** `192.168.1.10` **)** voglia inviare dati al **Server (** `192.168.1.20` **)** sulla stessa rete.

1. **Il tuo PC controlla la sua ARP Cache:**

- Prima di tutto, il tuo PC controlla la sua tabella ARP personale per vedere se conosce già l'indirizzo MAC del Server (`192.168.1.20`).
- **Se lo trova,** invia direttamente il frame al MAC trovato. Fine del processo.

2. **Se NON lo trova, invia un ARP Request (broadcast):**
   - Se la voce non è in cache, il tuo PC crea un pacchetto speciale chiamato **ARP Request**. Questo pacchetto contiene la domanda: *"Chi ha l'IP 192.168.1.20? Il suo MAC qual è?"*.
   - Questo pacchetto viene inviato in **broadcast** all'indirizzo MAC `FF:FF:FF:FF:FF:FF` . Questo significa che **TUTTI i dispositivi** sulla rete locale ricevono e processano questa richiesta.
3. **Il Server risponde con un ARP Reply (unicast):**
   - Tutti i dispositivi ricevono la richiesta, ma solo il dispositivo con l'IP `192.168.1.20` (il Server) risponderà.
   - Il Server crea un **ARP Reply** e lo invia **direttamente (unicast)** al MAC del tuo PC. La risposta dice: *"Ciao, sono io (192.168.1.20) e il mio indirizzo MAC è 00:1A:2B:3C:4D:5E".*
4. **Il tuo PC aggiorna la sua ARP Cache:**
   - Il tuo PC riceve la risposta, memorizza la coppia `IP <-> MAC` nella sua **ARP cache** e può ora inviare i dati al Server.

ARP Cache

- È una **tabella temporanea** nella memoria di **ogni dispositivo**.
- Le voci hanno una durata limitata (pochi minuti) e poi scadono. Questo perché gli indirizzi MAC possono cambiare (es., una scheda di rete sostituita) o un dispositivo può lasciare la rete.

> ⓘ **Si può visualizzare la propria ARP cache su Windows con il comando `arp -a` nel prompt dei comandi.**

> 🔗 **ARP, IP e indirizzo MAC**
>
> L'**indirizzo IP** viene (temporaneamente) associato all'**indirizzo MAC** fisico del dispositivo. L'indirizzo MAC è l'identificatore primario e permanente a livello di hardware.
>
> > ☰ **Facciamo un'analogia perfetta per chiarire il concetto:**

Pensa a una grande città (la tua rete locale/LAN):

1. **Indirizzo MAC (Media Access Control Address):**
   - È come **l'indirizzo fisico (civico) di un palazzo**. È unico, permanente e stampato sull'edificio.
   - Esempio: `Corso Italia, 123` - `08-16-05-ba-b9-30`

- **Non cambia mai.** Anche se il palazzo cambia proprietario, l'indirizzo fisico rimane lo stesso.

2. **Indirizzo IP (Internet Protocol Address):**
   - È come il **nome del proprietario attuale di un appartamento in quel palazzo**. È logico e può cambiare.
   - Esempio: `Mario Rossi` - `192.168.1.10`
   - **Può cambiare.** Se Mario Rossi si trasferisce, un nuovo proprietario (un nuovo dispositivo) si trasferirà in quell'appartamento e gli verrà assegnato lo stesso "nome" (IP). Oppure, Mario Rossi potrebbe prendere un altro IP dal DHCP.

Cosa fa ARP? Il Postino

Il protocollo **ARP** è il **postino molto efficiente** di questa città.

- Il postino (ARP) ha un pacco per **"Mario Rossi" (IP: 192.168.1.10)**. Ma per consegnarlo, ha bisogno di sapere in **quale palazzo specifico (MAC Address)** andare.
- Il postino non sa dove vive Mario Rossi, quindi va in mezzo alla piazza e **urla a tutti i palazzi** (broadcast): *"Hey, qualcuno sa dove abita Mario Rossi (192.168.1.10)?"*
- Tutti sentono, ma solo l'inquilino del **Corso Italia, 123** si affaccia alla finestra e urla: *"Sono io! Sono Mario Rossi! Il mio palazzo è questo (08-16-05-ba-b9-30)!"*
- Il postino prende nota su un foglietto (la **ARP cache**) che `Mario Rossi (192.168.1.10)` abita al `Corso Italia, 123 (08-16-05-ba-b9-30)` e gli consegna il pacco.
- La prossima volta che avrà un pacco per Mario, controllerà il suo foglietto e andrà direttamente al palazzo giusto, senza bisogno di urlare.

Network Access Layer

- Questo è lo strato più basso del modello TCP/IP.
- È responsabile della **trasmissione dei dati sul mezzo fisico** (rame, fibra, onde radio, ecc.)
- e prepara i pacchetti IP del livello superiore per il viaggio attraverso la rete locale.

Funzioni principali

1. **Data Framing (Incapsulamento):**
   - I pacchetti IP vengono incapsulati in **frame** adatti alla tecnologia di rete (Ethernet, Wi-Fi).
   - **Ogni frame ha un header (info mittente/destinatario) e un trailer (controllo errori).**
2. **Physical Addressing (Indirizzamento fisico):**
   - **Ogni dispositivo ha un indirizzo MAC univoco**.
   - **L'header del frame contiene MAC sorgente e MAC destinazione**.
3. **Error Detection (Rilevamento errori):**
   - **Il trailer include un CRC per controllare l'integrità.**
   - **Se il CRC non corrisponde, il frame viene scartato**.
4. **Access Control (chi può parlare e quando):**
   - Ethernet (cavo): usa **CSMA/CD** → **ascolta prima di trasmettere, gestisce collisioni.**
   - Wi-Fi (wireless): usa **CSMA/CA** → **ascolta e prenota il canale, riducendo le collisioni.**

Protocolli chiave

- **Ethernet:**
  - Standard per LAN cablate.
  - Definisce cavi, frame e metodo di accesso (CSMA/CD).
- **Wi-Fi (IEEE 802.11):**
  - Standard per WLAN wireless.
  - Gestisce modulazione radio, sicurezza (WPA2/WPA3) e accesso al mezzo (CSMA/CA).

Analogia con il Sistema Postale

- **Internet Layer (IP):** Decide la città, la via e il civico di destinazione finale (l'indirizzo IP).
- **Network Access Layer (Ethernet/Wi-Fi):** È il **postino** che:
  1. **Incapsula (Framing):** Mette la lettera in una busta standard delle poste.
  2. **Indirizzamento Fisico (MAC):** Sulla busta scrive il civico del **prossimo ufficio postale** (l'indirizzo MAC del router) a cui consegnare la lettera per farla proseguire.
  3. **Controllo Errori (CRC):** Applica un sigillo speciale sulla busta. Se il sigillo si rompe durante il trasporto, il prossimo ufficio postale sa che la lettera è stata danneggiata e la getta via.
  4. **Controllo Accesso:** Decide quando mettere la lettera nel camion per non farlo scontrare con altri camion in partenza.